Cordell Programming Language: A Lightweight Systems Language and Compiler Testbed

Author Name
Affiliation
email@example.com

Technical Report Draft, CPL v3.5

Abstract

Cordell Programming Language (CPL) is a deliberately restricted systems programming language and compiler infrastructure designed for low-level experimentation. Its syntax is close to C and assembly in intent, but it avoids implementing a C subset and instead uses a grammar suitable for an LL(1)-style parser. The language exposes explicit control over pointers, arrays, strings, entry points, system calls, inline assembly, and system-oriented annotations, while intentionally avoiding structures, classes, enums, garbage collection, memory safety guarantees, and production-language ecosystem assumptions. The compiler is organized as a multi-stage pipeline from preprocessing and tokenization to AST construction, semantic checking, HIR generation, SSA construction, high-level and low-level optimization, instruction selection, register allocation, assembly emission, and linking. This report documents the current language core, compiler pipeline, implemented static diagnostics, optimization passes, testing infrastructure, target status, preliminary microbenchmarks, limitations, and related work. The goal is not to present CPL as a verified or production-ready replacement for C or Rust, but as a compact experimental platform for studying compiler architecture, systems-language design, optimization, and diagnostics.

1 Introduction

Systems programming languages expose machine-level details while attempting to keep programs tractable. Mature languages such as C, C++, Rust, and Zig provide large ecosystems, but their full syntactic and semantic complexity makes them expensive targets for compiler experiments. Conversely, many teaching compilers are intentionally small but do not expose enough low-level mechanisms to study system-oriented code generation, platform-specific interfaces, and optimization under realistic constraints.

CPL addresses this gap as a small systems language and compiler testbed. The project originated from the need to learn compiler architecture and from an earlier idea of using a custom compiler inside a small operating-system project. That operating-system motivation is now mostly historical, but it still shaped CPL's design: the language stays close to C and assembly, minimizes runtime assumptions, and aims to remain portable to small or custom operating systems once an I/O API is supplied.

CPL is not a C subset. This is intentional. A faithful C-like subset would either require a substantially more complex parser and semantic model, or would create misleading expectations that the rest of C should also be supported. The current design instead favors an LL(1)-oriented grammar, explicit syntax for functions and pointer-like operations, and compiler-supported low-level constructs. For example, the project prefers explicit function forms over deeply nested C declarators for complex function pointer patterns.

The contribution of this report is a consolidated technical description of CPL v3.5:

- a concise language overview and design rationale;
- the current multi-stage compiler pipeline;
- the HIR, High LIR, Low LIR, and assembly form of a small example;
- the implemented static-analysis split between AST and IR diagnostics, including Z3-backed reasoning for selected conditions;
- the optimization passes currently documented for the compiler;
- preliminary microbenchmark results and their limitations;
- a related-work discussion positioning CPL among systems-language, educational-compiler, and verified-compiler projects.

2 Design Goals and Non-Goals

2.1 Goals

CPL is designed around five goals.

Low-level explicitness. The language is intended to stay close to assembly and C. It exposes pointers, explicit dereferencing, direct system calls, inline assembly, manual entry points, and annotations for low-level placement and control.

Small and readable core. The language deliberately avoids a large surface syntax. Instead of emulating all of C, CPL provides a compact grammar and a small set of orthogonal features. This keeps parsing and lowering simple enough for compiler experimentation.

Compiler experimentation. The compiler is a testbed for SSA construction, static diagnostics, constant propagation, constant folding, common subexpression elimination, loop-invariant code motion, peephole transformations, branch layout annotations, tail-recursion elimination, and register-allocation-sensitive low-level optimization.

System-oriented portability. CPL tries to minimize dependencies on a host standard library. The long-term intent is that the compiler and its runtime support can be ported to small or custom operating systems by providing basic I/O and platform APIs.

Educational value. The implementation is meant to expose a full compiler pipeline while staying small enough to inspect. This makes it useful for studying how language syntax, intermediate representations, optimizations, and backend constraints interact.

2.2 Non-Goals

CPL does not claim any of the following properties:

- formal verification of the compiler;
- memory safety or ownership checking;
- production readiness;
- source-level compatibility with C;
- self-hosting;
- a general-purpose standard library or package ecosystem;
- support for user-defined structures, classes, enums, or unions.

The absence of structures is an intentional restriction of the current core. It keeps the language focused on primitive values, pointers, arrays, strings, functions, and explicit low-level constructs.

3 Language Overview

3.1 Program Entry

A CPL program can use a `start` block as its static entry point. The entry block does not return a value with `return`; it must terminate through `exit`. An ordinary function can also become an entry point through the `@[entry]` annotation.

```
start() {
    exit 0;
}
```

Listing 1: Minimal CPL entry point.

The entry point may accept arguments, such as `argc` and `argv`, and can be annotated for platform-specific entry behavior. The `naked` annotation disables default entry and exit routines, which is useful for system-level code that must manage its own prologue and epilogue.

3.2 Types

CPL uses permissive static typing. Variables do not dynamically change type, widening conversions may be inserted implicitly, and narrowing conversions require an explicit `as` cast. The primitive integer and floating-point types include `i8`, `u8`, `i16`, `u16`, `i32`, `u32`, `i64`, `u64`, `f32`, `f64`, and the void-like function return type `i0`. Boolean-like logic follows the C convention: zero is false and non-zero is true.

3.3 Pointers, Strings, and Arrays

The language exposes pointer types through the `ptr` modifier. The `ref` keyword obtains a pointer to an object or string literal, and `dref` reads or writes through a pointer.

```
i32 x = 123;
ptr i32 p = ref x;
i32 y = dref p;
dref p = y + 1;
```

Listing 2: Pointer operations in CPL.

Strings and arrays are distinct built-in containers. A `str` allocates a NUL-terminated byte sequence, while `arr` declares a fixed-size array. String literals placed independently in code reside in a read-only section-like area and must be referenced explicitly.

```
str msg = "Hello world!";
arr xs[4, i32] = { 1, 2, 3, 4 };
ptr i8 p = ref "Hello, World!\n";
```

Listing 3: Strings and arrays.

3.4 Control Flow

CPL provides `if`, `while`, `loop`, and `switch`. A semicolon separates the condition from the body in conditional constructs. The `loop` construct represents an infinite loop unless terminated with `break` or annotated with `@[counter(n)]`. The `switch` construct supports fallthrough by default, while `@[no_fall]` inserts break-like behavior into cases.

```
if x > 0; {
    x -= 1;
} else {
    x = 0;
}

@[counter(10)] loop {
    x += 1;
}
```

Listing 4: Control flow.

3.5 Functions

Functions are declared with the `function` keyword. CPL supports prototypes, default arguments, local functions, function overloading with restrictions, generic functions, function pointers, lambdas without closure capture, and variadic arguments. Global functions use `glob` when their symbol name must be preserved for external linkage.

```
function add(i32 a, i32 b = 1) -> i32 {
    return a + b;
}

glob function exported(i32 x) -> i32;
```

Listing 5: Function forms.

Function pointers have no signature-level static information. This keeps them flexible but limits static checking, overload resolution, and default-argument insertion after a function is stored as a raw pointer.

3.6 System-Level Facilities

CPL provides two built-in low-level mechanisms: `syscall` and `asm`. A `syscall` expression is target-dependent and accepts a platform-specific argument list. Inline assembly supports argument substitution through numbered placeholders.

```
i32 a = 0;
i32 ret;
asm(a, ret) {
    "push rax",
    "mov rax, %0",
    "syscall",
    "mov %1, rax",
    "pop rax"
}
```

Listing 6: Inline assembly with placeholders.

Inline assembly is intentionally powerful and fragile. It is copied close to directly into the generated assembly after placeholder substitution, so the programmer must preserve registers and match the selected target syntax. The compiler does not optimize inside inline assembly blocks.

3.7 Annotations

Annotations extend the small grammar with low-level behavior. Table 1 summarizes the system-oriented annotations currently available in the implementation.

These annotations are not intended to make CPL a high-level language. Their role is to expose system-level control without expanding the core grammar.

4 Compiler Architecture

4.1 Pipeline

Figure 1 presents the current CPL compiler architecture. The frontend performs preprocessing, tokenization, AST construction, and early semantic checking. The middle-end then constructs HIR, converts it to SSA, runs SSA-level semantic checking based on symbolic reasoning and Z3, and applies high-level optimizations. The backend lowers the program to HLIR and target-sensitive low-level IR, performs scheduling and copy propagation, selects instructions, allocates registers, applies late peephole cleanup, and finally emits assembly for the system linker.

This organization deliberately separates source-level analysis, SSA construction, symbolic checking, high-level optimization, lowering, instruction selection, register allocation, and late cleanup. The separation makes the implementation suitable not only for code generation, but also for experiments with IR design, pass placement, and target-specific transformations.

4.2 Worked Example

Listing 7 shows a minimal program used to illustrate the compiler forms.

```
start() {
    i32 a = 1;
    i32 b = 2;
    exit a + b;
}
```

Listing 7: Input CPL program.

The corresponding HIR uses stack variables with an `s` suffix and temporaries with a `t` suffix.

```
{
fn _main0()
{
{
i32s %0 = alloc;
i32t %2 = i8n 1 as i32;
i32s %0 = i32t %2;
i32s %1 = alloc;
i32t %3 = i8n 2 as i32;
i32s %1 = i32t %3;
i32t %5 = i32s %0 + i32s %1;
u8t %4 = i32t %5 as u8;
exit u8t %4;
}
}
}
```

Listing 8: HIR form for Listing 7.

After lowering to High LIR, the program is represented as a basic-block sequence.

```
BB1: start
%2 = $1 as i32;
%0 = %2;
%3 = $2 as i32;
%1 = %3;
%5 = %0 + %1;
%4 = %5 as u8;
exit %4;
BB2: send
```

Listing 9: High LIR form.

A later Low LIR form contains target-dependent register choices.

```
BB1: start
rcx = $1;
rcx = rcx;
rdx = $2;
rdx = rdx;
```

Table 1: Selected CPL annotations.

Annotation	Purpose	Example
`naked`	Disable default entry and exit routines.	`@[naked] start()`
`align`	Align a local or global object.	`@[align(16)] glob i32 a;`
`section`	Place code or data in a target section.	`@[section(".data")]`
`address`	Place a function at a specific address.	`@[address(0x1000)]`
`entry`	Mark a function as an entry point.	`@[entry("_start")]`
`no_fall`	Insert break-like behavior in switch cases.	`@[no_fall] switch x;`
`straight`	Use linear switch selection.	`@[straight] switch x;`
`counter`	Generate a counted loop.	`@[counter(100)] loop`
`hot/cold`	Influence branch layout.	`@[cold] if cond;`
`register`	Bind a primitive variable to a register index.	`@[register(RAX)] i32 x;`

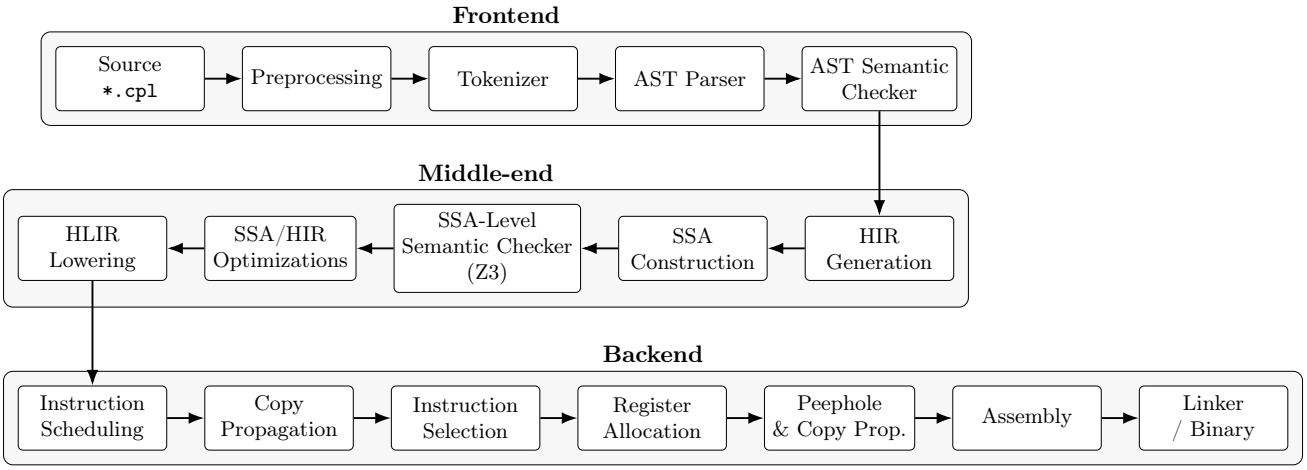

Figure 1: Graphical overview of the CPL compiler architecture. The pipeline is arranged in three phases to preserve readability in a two-column layout while retaining the execution order.

```
rax = rcx;
rax = rax + rdx;
rcx = rax;
rcx = rcx;
rdi = rcx;
exit rdi;
BB2: send
```

Listing 10: Low LIR form.

The final optimized Mach-O style assembly for this example collapses the computation to a short sequence.

```
; Compiled .asm file. File generated by
    CordellCompiler!
section .text
global _main0
_main0:
mov al, 1
add al, 2
mov ecx, eax
mov dil, cl
mov rax, 0x2000001
syscall
; BB2:
```

Listing 11: Generated assembly.

4.3 Backends and Target Status

The current stable target is x86_64 Mach-O in configurations that avoid the known function-inlining issues. An x86_64 Linux target is implemented but less tested. Planned targets include x86_32 Linux, x86_16 Linux, and i386 variants. The compiler can use `ld`, `clang`, or `gcc` as linkers; when external symbols are involved, `clang` or `gcc` are typically more convenient.

This report deliberately does not claim RISC-V as a working target and treats it as future or experimental work.

5 Static Diagnostics

CPL contains a two-level diagnostic subsystem. The first level operates on the AST and targets source-structure issues such as read-only variable updates, declaration problems, return mismatches, wrong argument counts, incompatible argument types, unused return values, illegal array access, duplicated branches, invalid function names, dead code, lossy implicit conversions, inefficient infinite loops, invalid `exit` usage, break statements without legal targets, invalid use of `i0`, unused expressions, invalid references, and suspicious alignment requests.

The second level is intentionally placed after SSA construction. At this point the compiler has explicit use-definition chains and phi nodes, so the checker can reason about values and path conditions in a more structured form than in the original HIR. Z3-backed symbolic execution is applied on SSA-form HIR to detect definite null dereferences, null values passed into dereferencing functions, possible null dereferences under path-dependent conditions, and constant branches. This ordering is important: the solver-facing analy-

sis works on SSA-form HIR rather than on pre-SSA HIR, because SSA simplifies symbolic value tracking and makes the generated constraints easier to relate to compiler variables.

The analysis is diagnostic rather than protective. It does not implement memory safety, ownership, or aliasing restrictions, and it does not prevent undefined behavior caused by dangling pointers, unsafe inline assembly, invalid external interfaces, or target-specific misuse.

```
start() {
    function foo(ptr i32 p) -> i32 {
        return dref p;
    }
    ptr i32 a = 0;
    foo(a);
}
```

Listing 12: Example that triggers SSA-level null-dereference diagnostics.

A representative diagnostic sequence for this example reports both that the return value of `foo(a)` is ignored and that `p` may be dereferenced while equal to null. More complex inputs additionally trigger path-sensitive reports, for example when one branch makes a pointer definitely null while another makes it only conditionally null.

5.1 Z3-Backed Symbolic Layer

The SSA-level checker is implemented as a symbolic layer over the HIR control-flow graph. The analyzer prepares HIR expressions as Z3 formulas, preserves symbolic names for compiler variables, and augments path conditions with phi-node constraints when control-flow edges enter SSA merge points. This design makes it possible to ask local reachability and value questions over the same representation that the optimizer sees.

The current wrapper accepts either parsed JSON or textual HIR dumps, can select a particular function, builds a CFG, and then dispatches solver-backed queries. Two query modes are especially useful for development: `label`, which checks whether a label can be reached under satisfiable path conditions, and `var-eq`, which checks whether a selected variable can, must, or cannot equal a given value. The wrapper also supports initial assignments, pointer-width configuration, strict parsing modes, and cached parsing/analysis artifacts. These features keep the solver path useful not only for compiler diagnostics, but also for investigating IR examples during pass development.

```
python3 z3_wrapper.py input.hir \
    --input-kind dump \
    --function _main \
    --ptr-bits 64 \
    var-eq %7 0
```

Listing 13: Representative Z3-wrapper query shape.

This subsystem should be understood as an experimental symbolic analysis component, not as a formal verification framework. Z3 is used to answer bounded path and value questions that are helpful for diagnostics such as null-dereference detection and unreachable-code reasoning. It does not by itself provide a complete semantic proof of CPL programs or of compiler transformations [3].

6 Optimization Passes

Table 2 summarizes the implemented optimization passes and their current optimization levels.

Table 2: Optimization passes implemented in the current CPL compiler.

Pass	IR level and role	Level
LICM	Works on SSA High IR; hoists loop-invariant expressions.	`-O2`
Peephole	Works on Low IR after instruction selection and register allocation.	`-O2`
CSE	Works on SSA IR using a DAG-based approach.	current optimized pipeline
DCE	Works during High IR CFG generation; unreachable code is marked unused and skipped by later lowering.	`-O0` and above
Constant propagation/folding	Works on High IR, then Low IR, then selected Low IR during assembly generation.	`-O1`
Function inlining	Works on High IR before SSA using a heuristic over source and destination control-flow properties.	`-O2`, known bugs
Tail-recursion elimination	Works on High IR and rewrites tail self-calls to loops.	`-O2`
Hot/cold placement	Implemented during AST-to-IR generation as annotation-driven branch layout.	`-O0` and above

6.1 Loop-Invariant Code Motion

LICM operates on SSA High IR. A loop body that repeatedly computes a constant expression such as `10 + 10` can have this computation moved before the loop, with phi nodes preserving loop-carried values.

```
start() {
    i32 d = 0;
    loop {
        i32 c = 10 + 10;
        d += c;
    }
}
```

Listing 14: LICM source pattern.

6.2 Peephole Optimization

Peephole optimization runs late, after instruction selection and register allocation. It removes redundant register-to-itself moves, simplifies decrement-and-branch sequences, and can replace zero materialization with xor-style idioms where applicable.

```
Before:
rax = rcx;
rax = rax - 1;
rcx = rax;
rdx = rcx;
cmp rdx, 0;
je lb11;
jne lb9;

After:
rcx = rcx - 1;
jne lb10;
```

Listing 15: Peephole effect on a counted empty loop.

6.3 PTRN: A Peephole Pattern DSL

The late peephole pass is not only a collection of hand-written rewrites. CPL also uses `PTRN`, a small domain-specific language for generating peephole optimization patterns. The motivation is practical: low-level instruction cleanup often consists of many small local rewrites, and encoding them in C by hand makes the pass difficult to extend. PTRN provides a declarative syntax for matching short Low LIR instruction sequences and replacing them with simpler or more efficient sequences.

A PTRN file consists of rewrite rules separated by `/`. The left-hand side describes a sequence to match; the right-hand side after `->` describes the replacement. Pattern objects abstract over concrete Low LIR operands: `areg_N` tracks a repeated abstract register, `aconst_N` tracks a repeated abstract constant, `mem_N` matches memory locations, and `obj` can match an arbitrary object. Conditions such as `[if:equals]`, `[if:arg2:zero]`, or `[if:arg2:mod2]` restrict a match, while actions can transform matched constants in the replacement.

```
; Remove a jump that immediately targets the next
    label.
jmp label_1
mklb label_1
->
mklb label_1
/

; Remove redundant self-copy.
mov obj_1, obj_2 [if:equals]
->
delete
/

; Prefer xor-zeroing over moving an immediate zero.
mov areg_1, const 0
->
xor areg_1, areg_1
/
```

Listing 16: Examples of PTRN peephole rules.

The generated C code is then used by the low-level peephole pass. This keeps the compiler backend extensible: adding a new local simplification can be done by adding a pattern rule rather than modifying the peephole engine directly. The approach is intentionally modest; PTRN does not replace global optimization or data-flow analysis. Its role is to make late instruction cleanup explicit, testable, and easier to maintain.

6.4 Function Inlining

The function-inlining pass works on High IR before SSA construction. It uses a heuristic that rewards small source functions, penalizes loop-heavy callees, and rewards call sites inside nested loops. The documented heuristic returns true when the score is at least 3. The pass exists, but the current implementation has known bugs and should not be presented as fully stable.

7 Examples and Use Cases

The current examples exercise core low-level features rather than large application workloads.

Table 3: Representative CPL programs.

Program	What it demonstrates
Hello World	Strings, pointers to string literals, `strlen`, inline assembly, direct syscall use, and entry-point termination.
CRC8	Global arrays, indexed memory access, loops, pointer arguments, integer operations, and exit-code calculation.
Brainfuck interpreter	Arrays, argument access, switch dispatch, `no_fall`, `straight`, loops, function calls, and byte-level tape manipulation.
Memory and file helpers	Header-style declarations, syscall wrappers, pointer arithmetic, and low-level I/O abstractions.

Listing 17 shows a compact Hello World program using a direct syscall wrapper.

```
function strlen(ptr i8 s) -> i64 {
    i64 l = 0;
    while dref s; {
        s += 1;
        l += 1;
    }
    return l;
}
```

```
function puts(ptr i8 s) -> i0 {
    asm (s, strlen(s)) {
        "push rdi",
        "push rsi",
        "push rdx",
        "mov rax, 33554436",
        "mov rdi, 1",
        "mov rsi, %0",
        "mov rdx, %1",
        "syscall",
        "pop rdx",
        "pop rsi",
        "pop rdi"
    }
}

start(i64 argc, ptr ptr i8 argv) {
    puts(ref "Hello, World!\n");
    exit 0;
}
```

Listing 17: Hello World with explicit syscall wrapper.

8 Testing Infrastructure

The compiler is tested with a phase-oriented framework rather than only through end-to-end examples. The core idea is to make each compiler stage observable: a test can stop after preprocessing, tokenization, AST generation, semantic analysis, HIR construction, SSA construction, DAG construction, constant-folding analysis, LIR construction, instruction selection, instruction planning, register allocation, peephole optimization, assembly generation, or assembly-level constant folding. This makes regressions easier to localize than in a purely black-box compiler test.

8.1 Integrated Pipeline Testing

The integration tester is built around a dedicated test driver that exercises the full compilation pipeline while allowing individual sections to be enabled or disabled at compile time. The active section is selected by a module argument. A typical invocation is:

```
python3 integrated_testing.py --run --module <name> \
    --debugger <lldb/gdb> \
    --test-code <path> \
    --extra-flags <flag>
```

Listing 18: Integrated testing command.

The current module set covers the main compiler phases:

For example, a basic assembly-generation regression test can be run by selecting the `asm` module. Memory-manager diagnostics can also be enabled through an extra compile-time flag and then post-processed by a leak analysis script.

```
python3 integrated_testing.py --run --module asm

python3 integrated_testing.py --run --module hir_dag
     \
    --test-code dummy_data/test2.cpl \
    --extra-flags DMEM_OPERATION_LOGS
python3 leaks.py --log-path <path_to_log>
```

Listing 19: Assembly and leak-oriented test invocations.

8.2 Module Tests

In addition to integration testing, the compiler has a module-level unit testing mode. A module test directory contains a C test harness and a dependency description. The harness computes the expected result for the selected compiler component, while the dependency file describes which compiler modules must be linked for the test. This mode is invoked as follows:

```
python3 module_testing.py --path <path>
```

Listing 20: Module testing command.

This separation is useful for testing compiler internals whose output is not naturally observable through a final executable. For example, AST construction, ownership-like metadata, data-flow sets, or register-allocation artifacts may be compared against expected textual output.

8.3 Test Oracle Format

CPL test files include an `OUTPUT` block that defines the expected compiler or runtime output. The framework also supports test flags placed at the beginning of a file. The most important flags are `: RUN_ASM :`, which builds and executes the generated assembly; `: RUN_ASM[args=...] :`, which executes the same test with several argument sets; `: BLOCK_TEST :`, which makes a failing test fail the whole run; `: BUG :`, which records an expected failing test; `: TEST_DEBUG :`, which runs under `gdb` or `lldb`; and `: LEAK_TRACE :`, which enables memory logging for leak localization.

The oracle language is intentionally tolerant where compiler dumps contain unstable identifiers. The marker `{X}` accepts an arbitrary substring, `{content}` requires a line to occur somewhere in the output, and `«content»` checks that a line contains a space-insensitive sequence. Runtime tests can also assert exit codes and select per-argument expected outputs through `@exit_code` and `@case_index`.

```
: RUN_ASM[args="apple"|args="banana"] :
function putc(i8 c) -> i0 {
    syscall(0x2000004, 1, ref c, 1);
}

@[entry("_main")]
function main(i32 argc, ptr ptr i8 argv) -> i0 {
    putc(argv[1][0]);
    exit 0;
}

:/ OUTPUT
@case_index=0
a
---
@case_index=1
b
---
/:
```

Listing 21: Runtime test with multiple argument cases.

Table 4: Compiler phases exposed by the integration testing framework.

Frontend	HIR / SSA	LIR / backend	Assembly
`preproc`	`hir`	`lir`	`asm`
`prep`	`hir_ssa`	`lir_constfold`	`asm_constfold`
`ast`	`hir_dag`	`lir_selector`	
`sem`	`hir_constfold`	`lir_instplan`	
		`lir_regalloc`	
		`lir_peephole`	

8.4 Representative Regression Programs

The supplied regression programs exercise both language features and backend behavior. They are also useful as benchmark kernels because the same `OUTPUT` blocks can request repeated measurements and generated-assembly line counts.

This testing setup does not amount to a proof of correctness. It is nevertheless important for a multi-stage compiler because it checks both internal forms and executable behavior, supports expected-failure tracking, and provides a practical way to detect regressions in individual passes before they are hidden by later lowering stages.

9 Preliminary Evaluation

The current evaluation should be read as a preliminary microbenchmark study rather than a broad performance claim. The chosen kernels are intentionally small and expose loop overhead, arithmetic simplification, and pointer-heavy traversal. They are useful for inspecting generated code and optimization effects, but they are not a substitute for large application benchmarks.

All available timings were collected on macOS Catalina 10.15.7 using an Intel i7-3650QM CPU at 2.4 GHz and 16 GB DDR3 memory. The compared tools were GCC 14.2.0, Apple Clang 12.0.0, CPL v3.4 for MACHO64, and rustc 1.87.0. GCC and Clang were tested at `-O0` and `-O3`, CPL at `-O0` and `-O3`, and Rust at `opt-level=0` and `opt-level=2`. Each timing was gathered five times after compilation. The measurements use different timing wrappers for CPL and for the C-family baselines, so the numbers should be interpreted conservatively. In particular, the empty-loop benchmark should be understood as a loop-overhead test: the C baseline uses `asm volatile` to prevent the compiler from deleting the loop.

9.1 Benchmarked Kernels

Listing 22 shows the counted empty loop used to evaluate loop overhead and late low-level simplification.

```
@[naked] start() {
    @[counter(1000000000)] loop {
    }
    exit 0;
}
```

Listing 22: Empty-loop benchmark kernel.

Listing 23 shows the Fibonacci kernel used to exercise simple arithmetic and loop-carried values.

```
start() {
    i32 a = 1;
    i32 b = 0;
    @[counter(1000000)] loop {
        i32 tmp = a;
        a = a + b;
        b = tmp;
    }
    exit b;
}
```

Listing 23: Fibonacci benchmark kernel.

Listing 24 shows the string-traversal kernel, which is more representative of pointer-based code generation.

```
start() {
    str msg = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+/";
    i64 outer = 0;
    u8 acc = 0;
    ptr i8 p = ref msg;

    while outer < 1000000; {
        p = ref msg;
        while dref p; {
            acc = (acc + dref p) & 0xFF;
            p += 1;
        }
        outer += 1;
    }

    exit acc;
}
```

Listing 24: String-traversal benchmark kernel.

9.2 Optimized Runtimes

Table 6 reports the optimized runtimes. Figure 2 visualizes the same measurements.

The results suggest that optimized CPL code is competitive with the optimized C-family baselines for the small kernels considered here, especially on the Fibonacci benchmark. The string-traversal kernel remains slower than Clang, GCC, and Rust, which indicates that pointer-heavy workloads are a more demanding target for the current backend.

9.3 Effect of Optimization within CPL

Figure 3 shows the effect of CPL optimization alone by comparing `-O0` and `-O3`. The empty loop benefits the most, which is consistent with the late peephole simplifications described earlier.

The optimized CPL assembly sizes reported for these kernels are compact: 11 lines for the empty loop, 17 lines for Fibonacci, and 84 lines for string traversal. These values are not a full proxy for performance, but they do suggest that the backend is capable of collapsing simple kernels to short instruction sequences.

Table 5: Representative CPL regression programs used by the current testing corpus.

Test program	Purpose
`01_simple_print.cpl`	String allocation, `strlen`, system call output, runtime output matching.
`02_count_to_billion.cpl`	Counted loop lowering, `naked` entry, exit-code checking, loop-overhead measurement.
`03_crc8.cpl`	Global lookup tables, pointer arguments, string scanning, indexed memory access, multi-argument runtime cases.
`04_arith_mix.cpl`–`06_function_call_hot.cpl`	Arithmetic kernels, branches, function-call overhead, hot-loop behavior, exit-code validation.
`07_table_sum_hot.cpl`–`09_fibonacci.cpl`	Table traversal, pointer/string scan, loop-carried values, integer overflow behavior, generated-code-size measurements.
`10_second_arith_mix.cpl`–`11_third_arith_mix.cpl`	Mixed arithmetic and function-call stress tests.
`12_brainfuck.cpl`	Larger interpreter-style program using arrays, switch dispatch, `no_fall`, `straight`, pointer movement, and output checking.
`13_simple_switch.cpl`	Small switch-dispatch regression with command-line arguments and exit-code assertion.

Table 6: Optimized benchmark runtimes in seconds. Lower is better.

Benchmark	CPL `-O3`	Clang `-O3`	GCC `-O3`	Rust opt-level=2
Empty loop	0.702	0.748	0.758	0.333
Fibonacci	0.412	0.417	0.412	0.335
String traversal	0.513	0.430	0.470	0.332

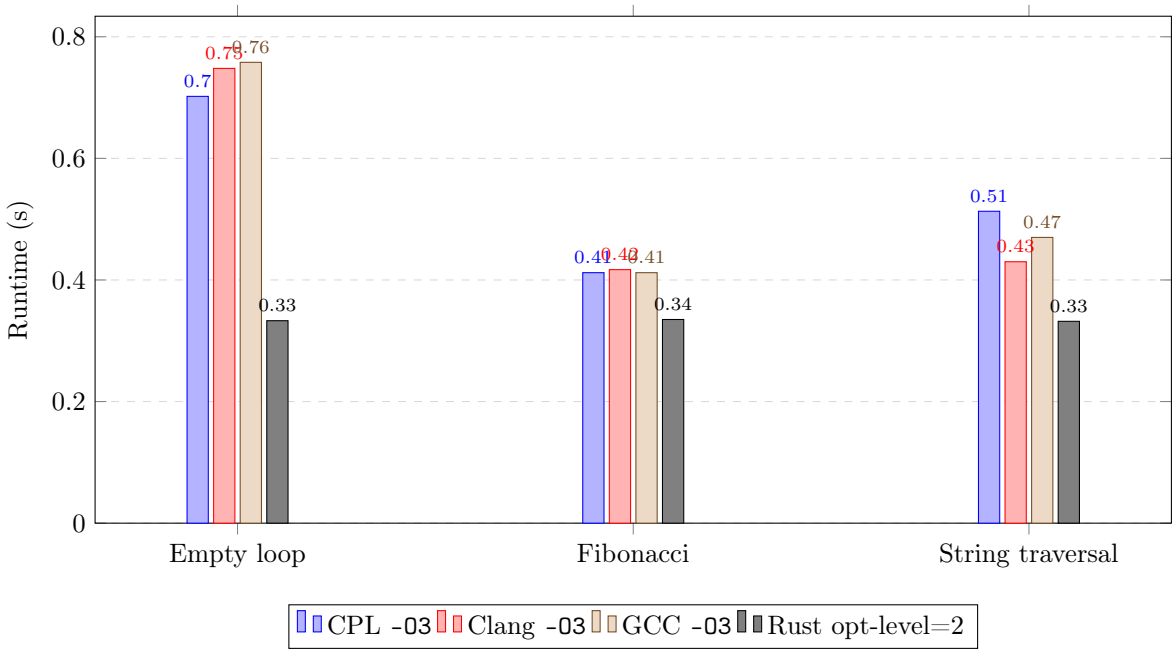

Figure 2: Optimized runtimes for the three benchmark kernels.

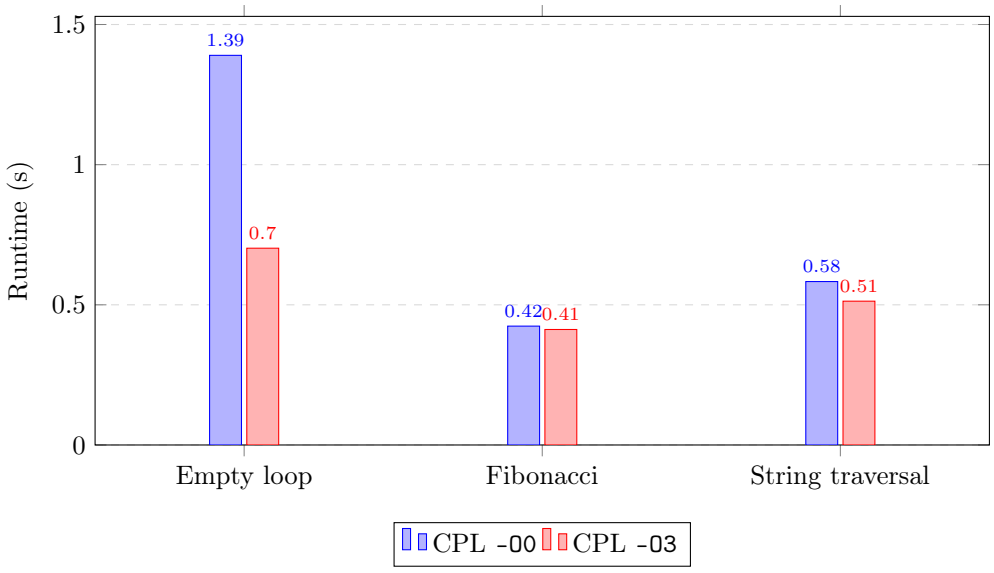

Figure 3: Optimization effect within CPL.

10 Limitations and Threats to Validity

The current report should be read with the following limitations in mind.

No formal semantics or proof. CPL does not currently provide a mechanized semantics, compiler-correctness proof, or translation validation.

No memory safety. The static analyzer can find selected problems, but CPL intentionally does not implement an ownership or borrowing discipline.

Limited target validation. The stable target is x86_64 Mach-O in configurations that avoid the known inlining issues. x86_64 Linux is implemented but less tested. Other targets should be treated as future or experimental.

Known function-inlining bugs. Function inlining exists, but it is not stable enough to be used as an unconditional compiler-quality claim.

Small benchmark suite. The benchmark suite consists of microbenchmarks and small interpreters. It does not include large applications, compiler self-hosting, broad target comparisons, or Linux x86_64 replication.

Version mismatch in evaluation. The report describes CPL v3.5, while the available benchmark dataset was collected for CPL v3.4 MACHO64. The benchmark data should therefore be treated as historical preliminary evidence rather than a complete v3.5 evaluation.

Inline assembly opacity. Inline assembly is copied into the output after placeholder substitution and is not optimized by the compiler. This can invalidate assumptions made by surrounding optimization passes if the programmer uses labels, jumps, or unpreserved registers in fragile ways.

11 Related Work

CPL is best understood as a small systems-language and compiler-construction testbed, not as a verified or production language.

Compiler frameworks. LLVM is a modular compiler infrastructure designed to support program analysis and transformation over a common low-level representation [1]. CPL is much smaller and does not attempt to provide a general reusable ecosystem, but it shares the idea that intermediate representations are central to optimization and backend experimentation.

SSA and optimization. CPL's SSA stage connects to the long-standing use of SSA as a representation for data-flow optimization [2]. The current implementation uses SSA as one stage in a larger HIR-to-LIR pipeline rather than as a complete formal IR specification. The symbolic analysis layer uses Z3, an SMT solver widely used for program analysis and verification-oriented tooling [3].

Verified systems languages and compilers. Cogent presents a restricted systems language with a self-certifying compiler and Isabelle/HOL artifacts [6]. Low* embeds verified low-level programming in F* and provides memory safety and verification support [7]. Jasmin targets efficient and formally verified cryptographic implementations [8]. CompCert is a formally verified C compiler with machine-checked semantic preservation guarantees [5]. CPL differs from all of these: it does not claim verified compilation, formal semantics, memory safety, or cryptographic security preservation.

Implementation reports for smaller languages. The Dino implementation report is close in genre: it documents language implementation decisions, VM/JIT design, and benchmark-driven implementation experience [9]. The improved GP 2 compiler work shows how implementation changes to an experimental language compiler can yield substantial performance improvements [10]. GNU epsilon provides a different example of a minimalistic language core intended for extensibility and analysis [11]. These works motivate reporting not only language syntax, but also implementation architecture and measured effects.

Educational compilers. Petit and small self-contained compiler courses emphasize educational compiler construction, staged language growth, and transparent implementation [12, 13]. CPL has an educational motivation, but differs by exposing system-level

constructs such as syscalls, inline assembly, low-level annotations, and explicit backend concerns.

Compiler testing. Csmith demonstrates the effectiveness of randomized differential testing for finding C compiler bugs [4]. CPL currently does not include a comparable fuzzing methodology. Such testing is an important direction for future work, especially because the compiler contains multiple lowering and optimization stages.

12 Future Work

The most important next steps are:

- strengthen Linux x86_64 target validation;
- repair known function-inlining bugs;
- publish reproducible benchmark scripts and raw data;
- extend the existing phase-oriented test suite with differential and randomized testing inspired by compiler-fuzzing work;
- document the IR formats more formally;
- improve diagnostics for inline assembly boundaries and target-specific calls;
- extend the PTRN rule set and add rule-level regression tests for peephole rewrites;
- expand the Z3 layer from ad-hoc investigations toward a more systematic SSA-level analysis API;
- expand the set of real programs beyond microbenchmarks and small interpreters.

Structures, classes, enums, and related user-defined aggregate types remain outside the current core by design. They may be studied separately, but this report treats their absence as part of CPL's deliberate restriction rather than a missing feature in the current language model.

13 Conclusion

CPL is a compact low-level language and compiler infrastructure for studying systems-language design and compiler implementation. It combines explicit low-level constructs, a restricted syntax, a multi-stage compiler pipeline, static diagnostics, SSA-based optimization, low-level backend passes, and target-specific assembly generation. Its value lies in being small enough to inspect and modify while still exposing problems that arise in real compilers: lowering boundaries, register allocation, branch layout, null reasoning, inline assembly opacity, and benchmark interpretation. The current implementation is not formally verified, memory-safe, production-ready, or self-hosting. Within those limits, CPL provides a useful experimental platform for compiler architecture and systems-programming research.

References

[1] Chris Lattner and Vikram Adve. LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization*, pages 75–88, 2004.

[2] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.

[3] Leonardo de Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340, 2008.

[4] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 283–294, 2011.

[5] The CompCert Project. The CompCert Verified Compiler. Project documentation, accessed 2026-05-08.

[6] Liam O'Connor, Christine Rizkallah, Zilin Chen, Sidney Amani, Japheth Lim, Yutaka Nagashima, Thomas Sewell, Alex Hixon, Gabriele Keller, Toby Murray, and Gerwin Klein. COGENT: Certified Compilation for a Functional Systems Language. arXiv:1601.05520, 2016.

[7] Jonathan Protzenko, Jean-Karim Zinzindohoue, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella-Beguelin, Antoine Delignat-Lavaud, Catalin Hritcu, Karthikeyan Bhargavan, Cedric Fournet, and Nikhil Swamy. Verified Low-Level Programming Embedded in F*. arXiv:1703.00053, 2017.

[8] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Gregoire, Vincent Laporte, and Paolo Torrini. The Jasmin Compiler Preserves Cryptographic Security. arXiv:2511.11292, 2025.

[9] Vladimir N. Makarov. Implementation of the Programming Language Dino – A Case Study in Dynamic Language Performance. arXiv:1604.01290, 2016.

[10] Graham Campbell, Jack Romo, and Detlef Plump. The Improved GP 2 Compiler. arXiv:2010.03993, 2020.

[11] Luca Saiu. GNU epsilon – an extensible programming language. arXiv:1212.5210, 2012.

[12] Raul Brajczewski Barbosa. Petit programming language and compiler. arXiv:2311.14443, 2023.

[13] Daniil Berezun and Dmitry Boulytchev. Reimplementing the Wheel: Teaching Compilers with a Small Self-Contained One. arXiv:2207.12698, 2022.